**SVEUČILIŠTE U ZAGREBU**

**FAKULTET ORGANIZACIJE I INFORMATIKE**

**V A R A Ž D I N**

**Valentino Mihalinac**

# Sustav za upravljanje Internetskom trgovinom

**PROJEKTNI RAD ZA KOLEGIJ BAZE PODATAKA 2**

**Mentori:**

Prof. dr. sc. Kornelije Rabuzin

Maja Cerjan, mag. educ. Inf.

**Varaždin, siječanj 2026.**

# Sadržaj

Sadržaj ..... 2
1. Uvod ..... 1
1.1. Korištene tehnologije i alati ..... 1
1.1.1. PostgreSQL ..... 1
1.1.2. PHP ..... 1
1.1.3. DataGrip ..... 2
1.1.4. Visual Studio Code ..... 3
1.1.5. XAMPP ..... 3
2. ERA model ..... 4
3. Opisi entiteta ..... 6
4. Primjeri upita ..... 8
4.1. Jednostavni upiti ..... 8
4.2. Složeni upiti ..... 8
4.3. Okidači ..... 9
5. Aplikacijsko sučelje za rad s bazom podataka ..... 12
6. Zaključak ..... 15
Popis literature ..... 16

# 1. Uvod

Cilj ovog projekta je implementacija baze podataka te korištenje osnovnih funkcionalnosti relacijskih baza podataka u programskom jeziku PHP. Glavna aplikacijska domena projekta je internetska trgovina.

Komunikacija s bazom podataka ostvarena je pomoću sučelja izrađenog u programskom jeziku PHP, dok su podaci pohranjeni u lokalnoj PostgreSQL bazi podataka. Razvijeno sučelje omogućuje dodavanje, izmjenu i brisanje podataka te demonstrira rad okidača definiranih u bazi podataka.

## 1.1. Korištene tehnologije i alati

### 1.1.1. PostgreSQL



Slika 1. Logotip PostgreSQL

PostgreSQL je moćan open source objektno-relacijski sustav za upravljanje bazama podataka koji koristi i proširuje SQL jezik te nudi napredne mogućnosti za sigurno pohranjivanje i obradu podataka. Razvoj PostgreSQL sustava započeo je još 1986. godine u sklopu POSTGRES projekta na Sveučilištu Kalifornije u Berkeleyu, a danas predstavlja jedan od najpouzdanijih sustava za upravljanje bazama podataka [1].

### 1.1.2. PHP

PHP je skriptni programski jezik koji se često koristi za izradu web aplikacija. U ovom projektu PHP je korišten za razvoj jednostavnog aplikacijskog sučelja koje omogućuje rad s bazom podataka i izvođenje osnovnih CRUD operacija. Komunikacija s bazom podataka ostvarena je s pomoću PDO (PHP Data Objects) sučelja.



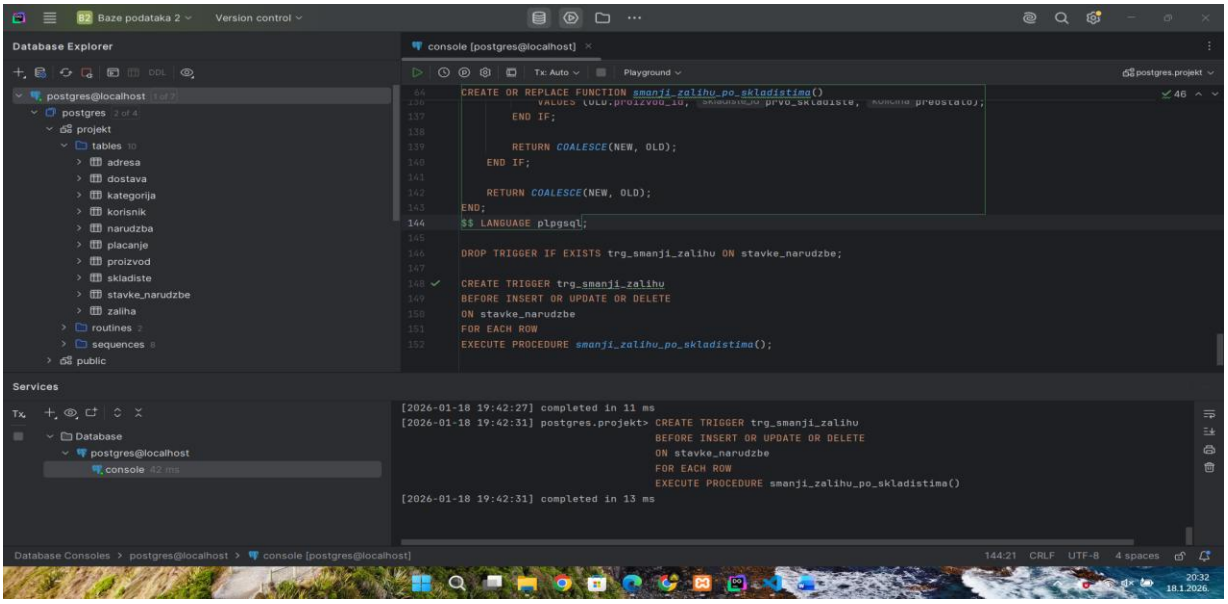
Slika 2. Logotip PHP

### 1.1.3. DataGrip

DataGrip je alat za administraciju baza podataka razvijen od strane tvrtke JetBrains. Omogućuje rad s različitim sustavima za upravljanje bazama podataka, pregled i uređivanje tablica, izvođenje SQL skripti te vizualizaciju strukture baze. U ovom projektu DataGrip je korišten za izradu baze podataka, administraciju sheme te testiranje SQL upita i okidača.



Slika 3. Logotip DataGrip




Slika 4. Snimka ekrana s prikazom baze, sheme, objekata u bazi, datuma i vremena

### 1.1.4. Visual Studio Code

Visual Studio Code je razvojno okruženje koje se koristi za izradu aplikacijskog sučelja u programskom jeziku PHP. Alat omogućuje rad s izvornim kodom, isticanje sintakse, korištenje proširenja za PHP te jednostavno povezivanje s lokalnim web poslužiteljem. U ovom projektu Visual Studio Code korišten je za razvoj i testiranje PHP skripti.



Slika 4. Logotip Visual Studio Code

### 1.1.5. XAMPP

XAMPP je paket alata koji uključuje web poslužitelj Apache i podršku za PHP. U ovom projektu XAMPP je korišten za pokretanje lokalnog web poslužitelja na kojem se izvršava aplikacijsko sučelje. Korištenjem XAMPP-a omogućeno je lokalno testiranje PHP aplikacije i povezivanje s PostgreSQL bazom podataka.

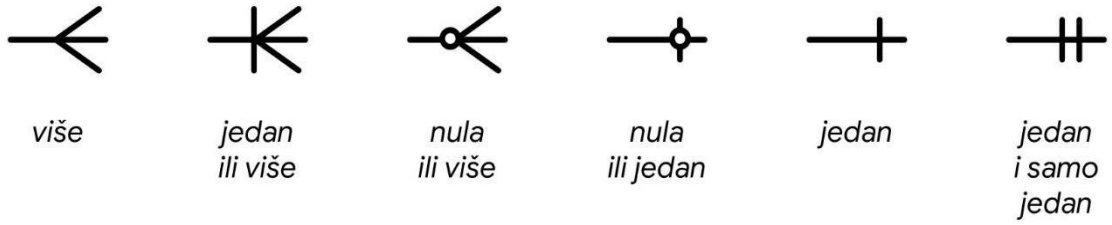


Slika 5. Logotip XAMPP

# 2. ERA model

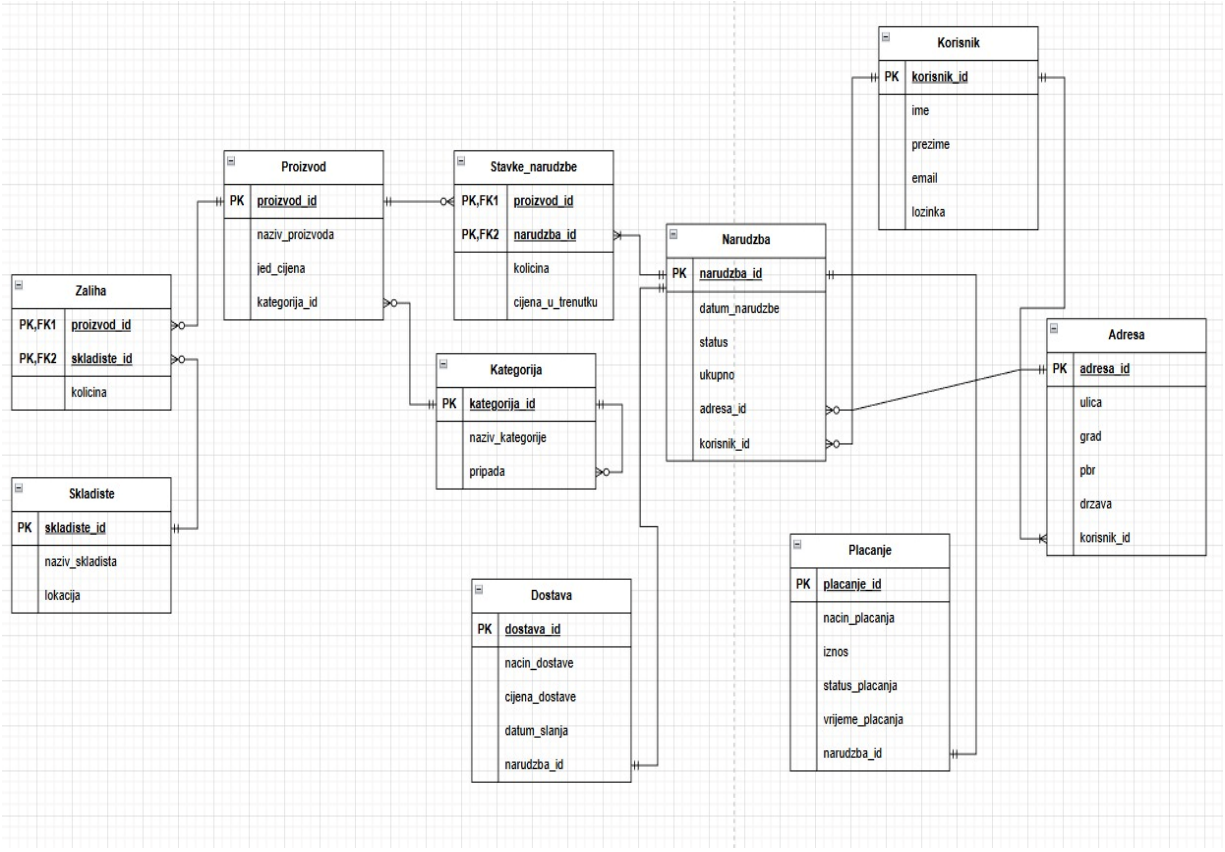
ERA-dijagram je slikovni prikaz podataka koji opisuje kako se podaci komuniciraju i međusobno povezuju. Bilo koji objekt, poput entiteta, atributa entiteta, skupova odnosa i drugih atributa odnosa, može se okarakterizirati uz pomoć ER dijagrama. [2] Veze u ERA modelu imaju svoju kardinalnost. Tako veze mogu biti sljedećih kardinalnosti:

- veza jedan na jedan (1:1)
- veza jedan na više (1:N)
- veza više na više (N:M)

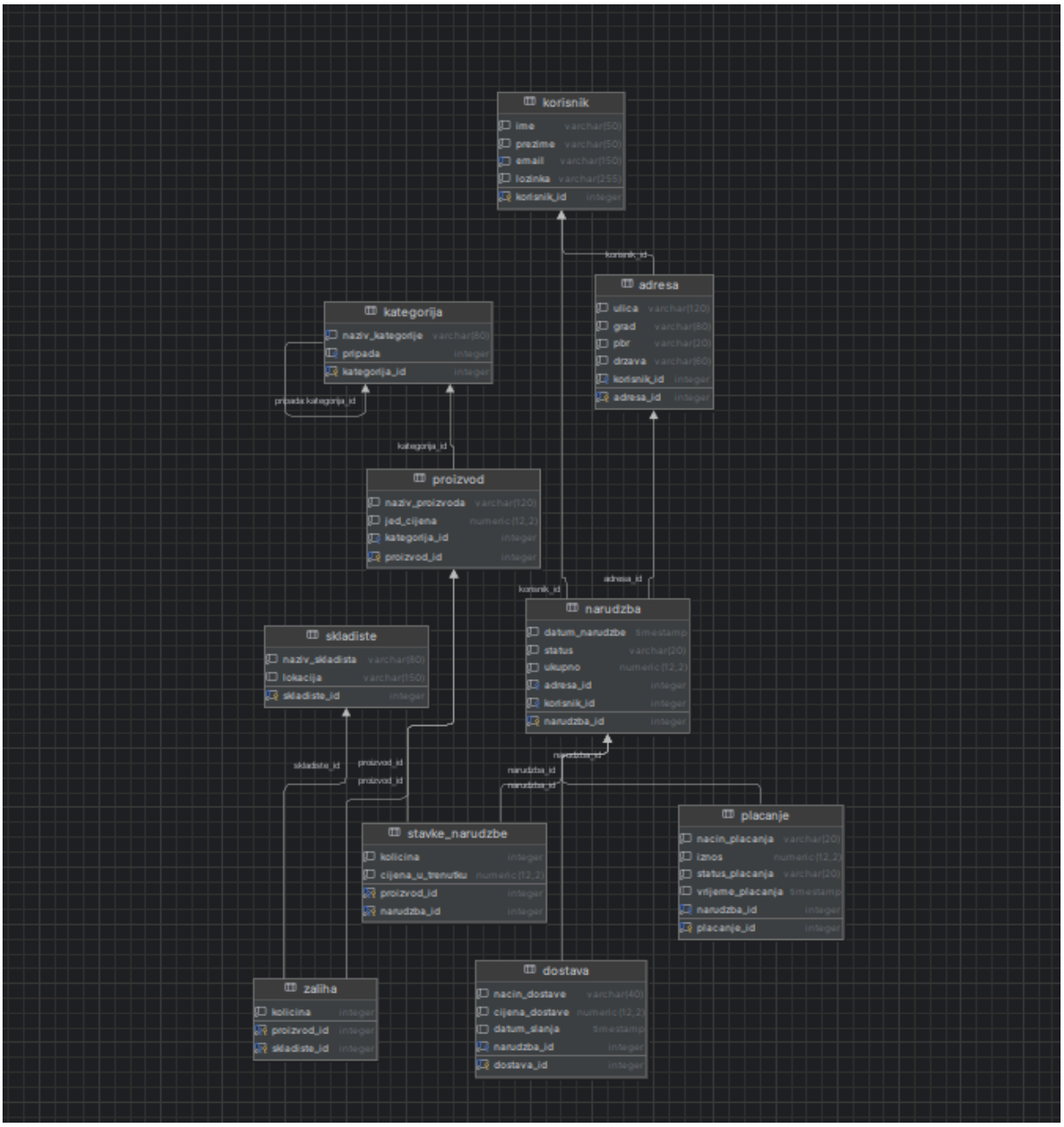



Slika 6. Simboli kardinalnosti veza u ERA dijagramu

Kako bi mogli ostvariti vezu N:M potrebno je dodati slabi entitet koji sadrži vanjske ključeve entiteta koji su u N:M vezi.




Slika 7. ERA model Internetske trgovine




Slika 8. Prikaz dijagrama iz DataGrip-a

# 3. Opisi entiteta

Entitet Korisnik sadrži osnovne podatke o korisnicima internetske trgovine. Atributi entiteta uključuju ime, prezime, adresu elektroničke pošte i lozinku. Primarni ključ entiteta je korisnik_id. Korisnik može imati jednu ili više adresa te može kreirati jednu ili više narudžbi.

Entitet Adresa sadrži podatke o adresama korisnika koje se koriste za dostavu narudžbi. Atributi entiteta uključuju ulicu, grad, poštanski broj i državu. Svaka adresa pripada točno jednom korisniku, dok jedan korisnik može imati više adresa. Entitet je povezan s entitetom Narudžba jer se prilikom izrade narudžbe odabire adresa dostave.

Entitet Kategorija služi za hijerarhijsku organizaciju proizvoda. Svaka kategorija može imati jednu nadkategoriju i više podkategorija, što predstavlja unarnu vezu tipa 1:N. Atributi entiteta uključuju naziv kategorije, dok je primarni ključ *kategorija_id*.

Entitet Proizvod sadrži podatke o proizvodima koji se prodaju u internetskoj trgovini. Atributi entiteta uključuju naziv proizvoda i jediničnu cijenu. Svaki proizvod pripada jednoj kategoriji, dok se njegova dostupnost prati kroz vezu s entitetom Zaliha.

Entitet Skladište predstavlja fizička skladišta u kojima se proizvodi pohranjuju. Atributi entiteta uključuju naziv skladišta i lokaciju. Jedno skladište može sadržavati više proizvoda, a svaki proizvod može biti pohranjen u više skladišta.

Entitet Zaliha predstavlja entitet koji nastaje zbog veze N:M između entiteta Proizvod i Skladište te omogućuje praćenje količine pojedinog proizvoda na određenom skladištu. Atribut entiteta je količina proizvoda. Primarni ključ entiteta sastoji se od kombinacije identifikatora proizvoda i skladišta.

Entitet Narudžba sadrži podatke o narudžbama koje su korisnici kreirali u sustavu. Atributi entiteta uključuju datum narudžbe, status narudžbe i ukupni iznos. Svaka narudžba povezana je s jednim korisnikom i jednom adresom dostave, a sastoji se od jedne ili više stavki narudžbe.

Entitet Stavke_narudžbe predstavlja slab entitet koji povezuje entitete Narudžba i Proizvod. Sadrži podatke o količini proizvoda i cijeni proizvoda u trenutku naručivanja. Primarni ključ entiteta sastoji se od kombinacije identifikatora narudžbe i proizvoda.

Entitet Plaćanje sadrži podatke o načinu i statusu plaćanja za pojedinu narudžbu. Atributi uključuju način plaćanja, iznos, status plaćanja i vrijeme plaćanja. Svaka narudžba ima najviše jedno plaćanje, čime je ostvarena veza tipa 1:1 između entiteta Narudžba i Plaćanje.

Entitet Dostava sadrži podatke o načinu dostave narudžbe, cijeni dostave i datumu slanja. Svaka dostava povezana je s točno jednom narudžbom, dok svaka narudžba ima najviše jednu dostavu. Time je ostvarena veza tipa 1:1 između entiteta Narudžba i Dostava.

# 4. Primjeri upita

## 4.1. Jednostavni upiti

```
SELECT COUNT(*)
FROM narudzba
WHERE status = 'Kreirano';
```
*---Vraća broj narudžbi koje su trenutno u statusu Kreirano*

```
SELECT SUM(kolicina)
FROM zaliha;
```
*---Vraća ukupnu količinu proizvoda koja se nalazi na skladištu*

```
SELECT MAX(jed_cijena)
FROM proizvod;
```
*---Vraća najveću jediničnu cijenu proizvoda*

## 4.2. Složeni upiti

```
SELECT korisnik.ime, korisnik.prezime,
    narudzba.datum_narudzbe,
    narudzba.status,
    narudzba.ukupno
FROM korisnik
JOIN narudzba ON korisnik.korisnik_id = narudzba.korisnik_id;
```
*---Vraća ime i prezime korisnika zajedno s datumom, statusom i ukupnim iznosom njihovih narudžbi*

```
SELECT proizvod.naziv_proizvoda,
    proizvod.jed_cijena,
    kategorija.naziv_kategorije
```

```
FROM proizvod
JOIN kategorija ON proizvod.kategorija_id = kategorija.kategorija_id;
---Vraća naziv proizvoda, njegovu cijenu i naziv kategorije kojoj pripada


 SELECT s.naziv_skladista,
      k.naziv_kategorije,
      SUM(z.kolicina) AS ukupno_kolicina
FROM skladiste s
JOIN zaliha z ON z.skladiste_id = s.skladiste_id
JOIN proizvod p ON p.proizvod_id = z.proizvod_id
JOIN kategorija k ON k.kategorija_id = p.kategorija_id
GROUP BY s.naziv_skladista, k.naziv_kategorije
ORDER BY s.naziv_skladista, ukupno_kolicina DESC;
---Vraća ukupnu količinu proizvoda po skladištu i kategoriji
```

## 4.3. Okidači

```
CREATE OR REPLACE FUNCTION izracun_ukupno_narudzbe()
RETURNS trigger
AS
$$
DECLARE
   v_narudzba_id INTEGER;
   novi_ukupno NUMERIC(12,2);
BEGIN
   v_narudzba_id := COALESCE(NEW.narudzba_id, OLD.narudzba_id);

   SELECT COALESCE(SUM(kolicina * cijena_u_trenutku), 0)
   INTO novi_ukupno
   FROM stavke_narudzbe
   WHERE narudzba_id = v_narudzba_id;

   UPDATE narudzba
   SET ukupno = novi_ukupno
   WHERE narudzba_id = v_narudzba_id;

   RAISE NOTICE 'Ukupni iznos narudžbe % iznosi % EUR.',
      v_narudzba_id, novi_ukupno;

   RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_izracun_ukupno
AFTER INSERT OR UPDATE OR DELETE
ON stavke_narudzbe
FOR EACH ROW
EXECUTE PROCEDURE izracun_ukupno_narudzbe();
```

*-- Izračunava i ažurira ukupni iznos narudžbe nakon dodavanja ili izmjene stavke narudžbe (insert,update,delete)*

```
[2026-01-11 14:15:50] postgres.projekt> INSERT INTO stavke_narudzbe (proizvod_id, narudzba_id, kolicina, cijena_u_trenutku)
                                        VALUES (1, 2, 1, 1200.00)
Ukupni iznos narudžbe 2 iznosi 3000.00 EUR.
```

Slika 9. Testiranje prvog okidača

```
CREATE OR REPLACE FUNCTION smanji_zalihu_po_skladistima()
RETURNS trigger
AS
$$
DECLARE
   preostalo INTEGER;
   r RECORD;
   uzmi INTEGER;
   prvo_skladiste INTEGER;
BEGIN
   IF TG_OP = 'INSERT' OR (TG_OP = 'UPDATE' AND NEW.kolicina > OLD.kolicina)
THEN

      IF TG_OP = 'INSERT' THEN
         preostalo := NEW.kolicina;
      ELSE
         preostalo := NEW.kolicina - OLD.kolicina;
      END IF;

      FOR r IN
         SELECT skladiste_id, kolicina
         FROM zaliha
         WHERE proizvod_id = NEW.proizvod_id
           AND kolicina > 0
         ORDER BY skladiste_id
         FOR UPDATE
      LOOP
         EXIT WHEN preostalo <= 0;

         uzmi := LEAST(preostalo, r.kolicina);

         UPDATE zaliha
         SET kolicina = kolicina - uzmi
         WHERE proizvod_id = NEW.proizvod_id
           AND skladiste_id = r.skladiste_id;

         preostalo := preostalo - uzmi;
      END LOOP;

      IF preostalo > 0 THEN
```

```
            RAISE EXCEPTION
               'Nema dovoljno zalihe za proizvod %. Nedostaje %.',
               NEW.proizvod_id, preostalo;
         END IF;

         RETURN NEW;
      END IF;

      IF TG_OP = 'DELETE' OR (TG_OP = 'UPDATE' AND NEW.kolicina < OLD.kolicina)
THEN

         SELECT MIN(skladiste_id)
         INTO prvo_skladiste
         FROM skladiste;

         IF TG_OP = 'DELETE' THEN
            preostalo := OLD.kolicina;
         ELSE
            preostalo := OLD.kolicina - NEW.kolicina;
         END IF;

         UPDATE zaliha
         SET kolicina = kolicina + preostalo
         WHERE proizvod_id = OLD.proizvod_id
           AND skladiste_id = prvo_skladiste;

         IF NOT FOUND THEN
            INSERT INTO zaliha (proizvod_id, skladiste_id, kolicina)
            VALUES (OLD.proizvod_id, prvo_skladiste, preostalo);
         END IF;

         RETURN COALESCE(NEW, OLD);
      END IF;

      RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_smanji_zalihu
BEFORE INSERT OR UPDATE OR DELETE
ON stavke_narudzbe
FOR EACH ROW
EXECUTE PROCEDURE smanji_zalihu_po_skladistima();
```

*--Okidač za ažuriranje zalihe proizvoda prilikom dodavanja, izmjene i brisanja stavki narudžbe*

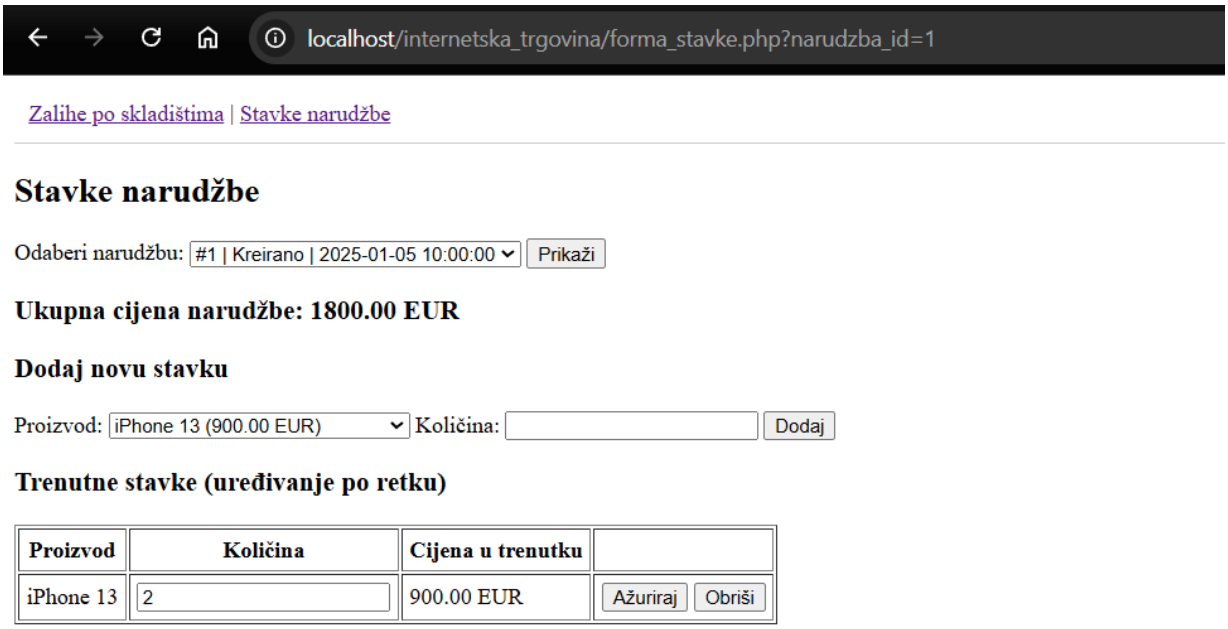
```
330
331  INSERT INTO stavke_narudzbe
332  (proizvod_id, narudzba_id, kolicina, cijena_u_trenutku)
333  VALUES ( proizvod_id 2,  narudzba_id 3,  kolicina 16,  cijena_u_trenutku 150.00);
334

[P0001] ERROR: Nema dovoljno zalihe za proizvod 2. Nedostaje 1.
  Where: PL/pgSQL function smanji_zalihu_po_skladistima() line 28 at RAISE
```

Slika 10. Testiranje drugog okidača

# 5. Aplikacijsko sučelje za rad s bazom podataka

Sučelje je zamišljeno iz perspektive administratora sustava, koji ima uvid u narudžbe, njihove stavke te stanje zaliha proizvoda po skladištima, te može upravljati prikazanim podacima.




Slika 11. Prikaz forme Stavke narudžbe

Prva forma prikazuje rad s entitetom stavke_narudžbe, koji predstavlja vezu između narudžbe i proizvoda. Forma omogućuje odabir postojeće narudžbe te pregled svih stavki koje toj narudžbi pripadaju.

Na vrhu forme nalazi se padajući izbornik za odabir narudžbe, pri čemu se za svaku narudžbu prikazuje njezin identifikator, status i datum kreiranja. Nakon odabira narudžbe, forma

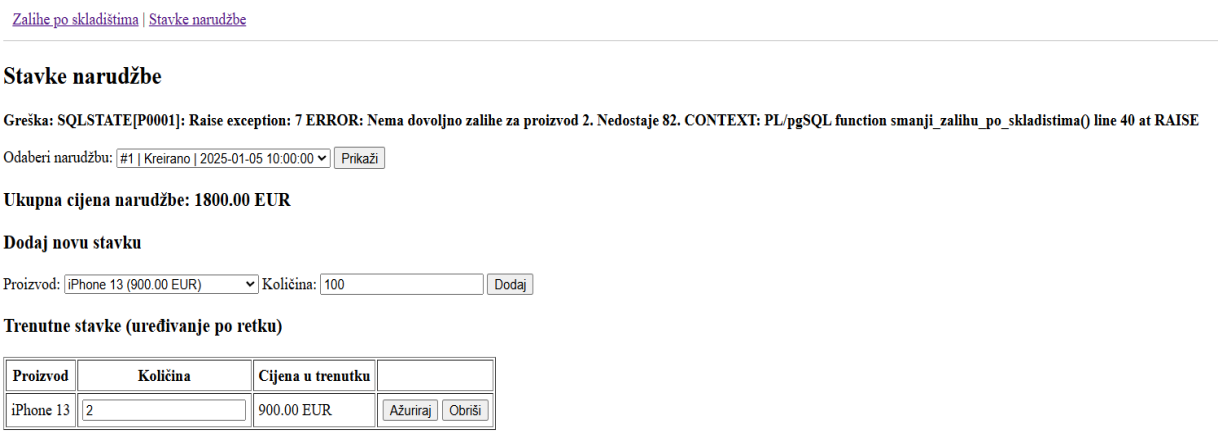
dohvaća i prikazuje ukupni iznos narudžbe, koji se automatski izračunava pomoću okidača u bazi podataka.

Forma omogućuje sljedeće funkcionalnosti:

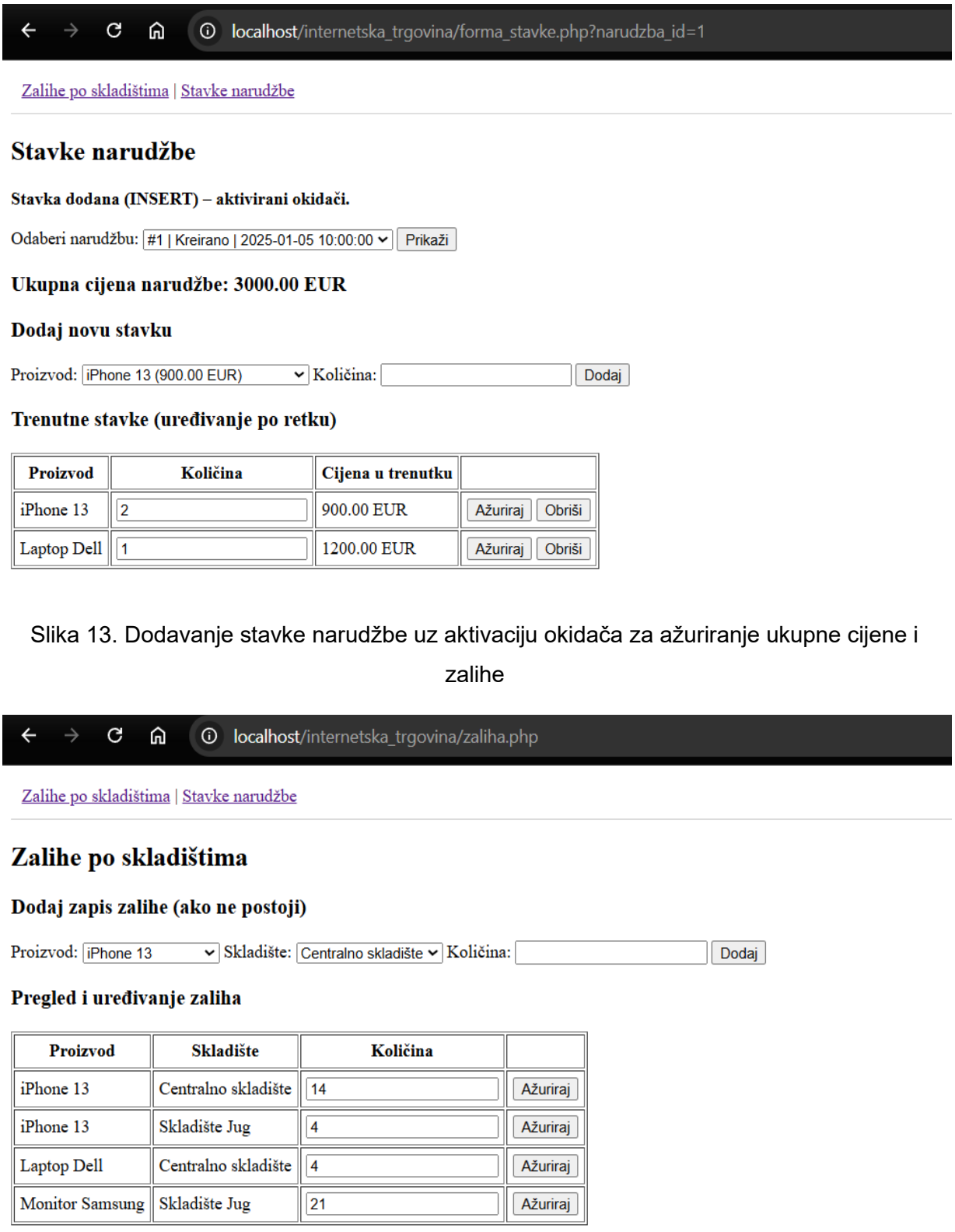
- dodavanje nove stavke narudžbe (odabir proizvoda i količine),
- izmjenu količine postojeće stavke,
- brisanje stavke iz narudžbe.

Dodavanjem nove stavke u narudžbu izvršava se INSERT operacija nad tablicom stavke_narudžbe. Prilikom tog unosa automatski se aktivira okidač za smanjenje zaliha proizvoda po skladištima te okidač za ponovni izračun ukupnog iznosa narudžbe. Prilikom izmjene količine postojeće stavke koristi se UPDATE operacija. Ova operacija također aktivira okidače koji prilagođavaju stanje zaliha i ponovno izračunavaju ukupnu cijenu narudžbe. Na taj način onemogućeno je povećavanje količine iznad raspoložive zalihe proizvoda.

Brisanjem stavke iz narudžbe izvršava se DELETE operacija, pri čemu se kroz okidač vraća odgovarajuća količina proizvoda natrag na zalihu te se ažurira ukupni iznos narudžbe.




Slika 12. Aktivacija okidača za provjeru zalihe kroz aplikacijsko sučelje




Slika 13. Dodavanje stavke narudžbe uz aktivaciju okidača za ažuriranje ukupne cijene i zalihe




Slika 14. Prikaz forme Zalihe po skladištima

Druga forma omogućuje pregled trenutačnog stanja zaliha proizvoda po skladištima. Forma dohvaća podatke iz tablice zaliha te prikazuje za svaki proizvod količinu dostupnu u pojedinom skladištu.

Ova forma služi za pregled, dodavanje i ažuriranje podataka, a koristi se kako bi se demonstrirao učinak okidača za smanjenje i povećanje zaliha prilikom rada s narudžbama. Nakon dodavanja, izmjene ili brisanja stavke narudžbe, promjene u zalihama su odmah vidljive u ovoj formi.

# 6. Zaključak

U okviru projekta izrađena je baza podataka za sustav internetske trgovine koja obuhvaća osnovne entitete i njihove odnose. Razvijeni ERA model implementiran je u sustavu PostgreSQL, pri čemu su definirane tablice, veze i okidači koji osiguravaju ispravno funkcioniranje baze podataka.

Izrađeno aplikacijsko sučelje u programskom jeziku PHP omogućuje osnovne operacije dodavanja, izmjene i brisanja podataka te provjeru rada okidača i poslovnih pravila u bazi podataka. Prije implementacije aplikacijskog sučelja bilo je potrebno proučiti osnove rada u programskom jeziku PHP, kao i postupak povezivanja PHP aplikacije s PostgreSQL bazom podataka.

# Popis literature

[1] PostgreSQL: https://www.postgresql.org/about/ dostupno 11.01.2026.

[2] Model odnosa entiteta (ER): https://www.w3schools.in/dbms/er-model dostupno 11.01.2026.